LAPORAN PRAKTIKUM
JOBSHEET 12 - POLIMORFISME



Disusun Oleh:

Nama	:	Mohammad Hafidz Rafi' Rabbani
NIM	:	254107020084
Kelas	:	2H
No. Absen	:	14
Repository	:	github.com/hafidzrafi/java-oop

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026

Daftar Isi

1. Tujuan 3

2. Praktikum 3

2.1. Percobaan 1: Bentuk Dasar Polimorfisme 3

2.1.1. Pertanyaan 7

2.2. Percobaan 2: Virtual Method Invocation 9

2.2.1. Pertanyaan 10

2.3. Percobaan 3: Heterogeneous Collection 12

2.3.1. Pertanyaan 13

2.4. Percobaan 4: Polymorphic Arguments, instanceof, dan Object Casting 15

2.4.1. Pertanyaan 17

3. Tugas 19

3.1. Tugas Mandiri: Sistem Pertahanan Plants vs Zombies 19

1. Tujuan

- Memahami konsep dasar dan bentuk-bentuk polimorfisme (subtype polymorphism) dalam pemrograman berorientasi objek.
- Menganalisis mekanisme kerja dan implementasi Virtual Method Invocation (dynamic method dispatch) di level runtime JVM.
- Menerapkan konsep polimorfisme dalam perancangan koleksi data heterogen (Heterogeneous Collection).
- Menerapkan polimorfisme pada argumen metode (Polymorphic Arguments) serta memahami mekanisme Runtime Type Identification (instanceof) dan eksplisit object downcasting.
- Menyimpan dan mendokumentasikan seluruh source code pekerjaan pada repositori GitHub resmi: https://github.com/hafidzrafi/java-oop.

2. Praktikum

2.1. Percobaan 1: Bentuk Dasar Polimorfisme

Pada percobaan pertama, diperkenalkan konsep dasar polimorfisme (**subtype polymorphism**). Diimplementasikan superclass `Employee` yang mewariskan atribut `name` dan method `getEmployeeInfo()`. Class `InternshipEmployee` mewarisi `Employee`, sedangkan `PermanentEmployee` mewarisi `Employee` sekaligus mengimplementasikan interface `Payable`. Selain itu, class `ElectricityBill` mengimplementasikan interface `Payable` sebagai entitas non-karyawan yang dapat dibayarkan.

Melalui class `Tester1`, dibuktikan bahwa **reference variable** bertipe superclass `Employee` dapat menampung referensi objek dari subclass-nya (`pEmp` dan `iEmp`), serta **reference variable** bertipe interface `Payable` dapat menampung objek konkret dari kelas pengimplementasinya (`pEmp` dan `eBill`) melalui mekanisme **implicit upcasting**.

```
// Employee.java & Payable.java
package jobsheet.jobsheet12.src.experiments;

public class Employee {
    protected String name;

    public String getEmployeeInfo() {
```

```
        return "Name = " + name;
    }
}

public interface Payable {
    public int getPaymentAmount();
}
```

```
// InternshipEmployee.java & PermanentEmployee.java
package jobsheet.jobsheet12.src.experiments;

public class InternshipEmployee extends Employee {
    private int length;

    public InternshipEmployee(String name, int length) {
        this.name = name;
        this.length = length;
    }

    public int getLength() { return length; }
    public void setLength(int length) { this.length = length; }

    @Override
    public String getEmployeeInfo() {
        String info = super.getEmployeeInfo() + "\n";
        info += "Registered as internship employee for " + length + " month/s\n";
        return info;
    }
}

public class PermanentEmployee extends Employee implements Payable {
    private int salary;

    public PermanentEmployee(String name, int salary) {
        this.name = name;
        this.salary = salary;
    }

    public int getSalary() { return salary; }
    public void setSalary(int salary) { this.salary = salary; }

    @Override
    public int getPaymentAmount() {
        return (int) (salary + 0.05 * salary);
    }

    @Override
    public String getEmployeeInfo() {
        String info = super.getEmployeeInfo() + "\n";
        info += "Registered as permanent employee with salary " + salary + "\n";
```

```
        return info;
    }
}
```

```
// ElectricityBill.java & Tester1.java
package jobsheet.jobsheet12.src.experiments;

public class ElectricityBill implements Payable {
    private int kwh;
    private String category;

    public ElectricityBill(int kwh, String category) {
        this.kwh = kwh;
        this.category = category;
    }

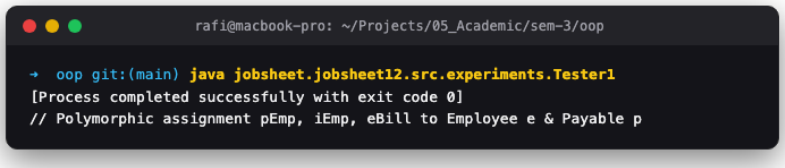
    @Override
    public int getPaymentAmount() { return kwh * getBasePrice(); }

    public int getBasePrice() {
        int bprice = 0;
        switch (category) {
            case "R-1": bprice = 100; break;
            case "R-2": bprice = 200; break;
        }
        return bprice;
    }

    public String getBillInfo() {
        return "kWH = " + kwh + "\n" + "Category = " + category + "(" + getBasePrice()
+ " per kWH)\n";
    }
}

public class Tester1 {
    public static void main(String[] args) {
        PermanentEmployee pEmp = new PermanentEmployee("Dedik", 500);
        InternshipEmployee iEmp = new InternshipEmployee("Sunarto", 5);
        ElectricityBill eBill = new ElectricityBill(5, "A-1");
        Employee e;
        Payable p;

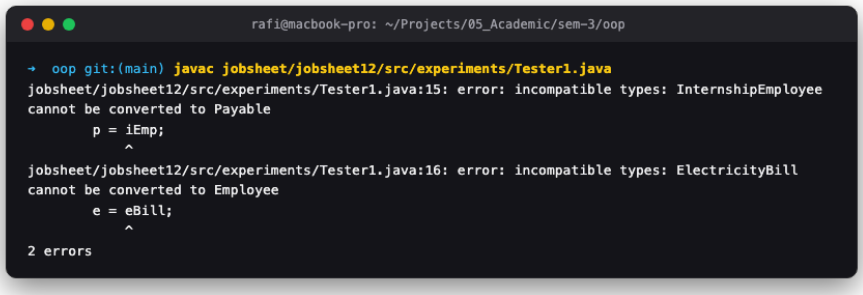
        e = pEmp;
        e = iEmp;
        p = pEmp;
        p = eBill;
    }
}
```


rafi@macbook-pro: ~/Projects/05_Academic/sem-3/oop
→ oop git:(main) java jobsheet.jobsheet12.src.experiments.Tester1
[Process completed successfully with exit code 0]
// Polymorphic assignment pEmp, iEmp, eBill to Employee e & Payable p

2.1.1. Pertanyaan

1. Class apa sajakah yang merupakan turunan dari class `Employee`?
    - Class yang merupakan turunan langsung dari superclass `Employee` adalah `PermanentEmployee` dan `InternshipEmployee`.
2. Class apa sajakah yang mengimplementasikan interface `Payable`?
    - Class yang mengimplementasikan interface `Payable` adalah `PermanentEmployee` dan `ElectricityBill`.
3. Perhatikan class `Tester1`, baris 10 dan 11. Mengapa `e`, bisa diisi dengan objek `pEmp` (merupakan objek dari class `PermanentEmployee`) dan objek `iEmp` (merupakan objek dari class `InternshipEmployee`)?
    - Variabel `e` merupakan **reference variable** yang dideklarasikan dengan tipe statis (**compile-time type**) `Employee`. Di JVM, variabel bertipe referensi pada stack frame menyimpan **reference value** (pointer memori ke objek konkret di Heap). Karena `PermanentEmployee` dan `InternshipEmployee` mewarisi superclass `Employee` (memiliki relasi **IS-A**), kompiler Java mengizinkan penugasan alamat memori objek subclass ke variabel bertipe superclass melalui mekanisme **widening reference conversion** (**implicit upcasting**).
4. Perhatikan class `Tester1`, baris 12 dan 13. Mengapa `p`, bisa diisi dengan objek `pEmp` (merupakan objek dari class `PermanentEmployee`) dan objek `eBill` (merupakan objek dari class `ElectricityBill`)?
    - Variabel `p` merupakan **reference variable** bertipe interface `Payable` (**contract type**). Dalam sistem tipe Java, **interface reference** dapat menyimpan **reference value** dari setiap instance objek kelas yang mengimplementasikan (`implements`) kontrak interface tersebut. Karena `PermanentEmployee` dan `ElectricityBill` sama-sama mengimplementasikan interface `Payable`, pointer alamat dari kedua objek tersebut sah ditugaskan ke variabel `p`.
5. Coba tambahkan sintaks `p = iEmp; e = eBill;` pada baris 14 dan 15 (baris terakhir dalam method `main`)! Apa yang menyebabkan terjadinya error?
    - Kompilasi gagal dengan pesan error saat **compile-time**:

- `error: incompatible types: InternshipEmployee cannot be converted to Payable`
- `error: incompatible types: ElectricityBill cannot be converted to Employee`



Penyebabnya adalah kegagalan **static type checking** oleh kompiler Java. Pada hierarki tipe data, class `InternshipEmployee` tidak mengimplementasikan interface `Payable`, dan class `ElectricityBill` tidak memiliki relasi pewarisan (**IS-A**) dengan superclass `Employee`. Karena tidak ada kompatibilitas tipe data pada hierarki, penugasan referensi tersebut melanggar aturan sistem tipe Java dan ditolak oleh compiler.

6. Buat kesimpulan tentang konsep/bentuk dasar polimorfisme!
   - Bentuk dasar polimorfisme (**subtype polymorphism**) memungkinkan sebuah **reference variable** bertipe superclass atau interface untuk menampung **reference value** (pointer ke memori Heap) dari berbagai objek konkret subclass (hasil pewarisan `extends`) maupun kelas pengimplementasi (hasil `implements`). Secara arsitektur, kode pemanggil dapat memperlakukan beragam objek konkret yang berbeda secara seragam selama objek-objek tersebut terikat dalam satu kontrak tipe dasar yang sama.

2.2. Percobaan 2: Virtual Method Invocation

Percobaan kedua menguji konsep **Virtual Method Invocation** (VMI). Objek `PermanentEmployee` diinstansiasi dan alamat referensinya disimpan ke dalam variabel lokal `pEmp` bertipe `PermanentEmployee` serta di-**upcast** ke variabel `e` bertipe `Employee`. Selanjutnya dilakukan pemanggilan method `getEmployeeInfo()` dari kedua variabel tersebut.

```java
// Tester2.java
package jobsheet.jobsheet12.src.experiments;

public class Tester2 {
    public static void main(String[] args) {
        PermanentEmployee pEmp = new PermanentEmployee("Dedik", 500);
        Employee e = pEmp;

        System.out.println("" + e.getEmployeeInfo());
        System.out.println("----------------------------");
        System.out.println("" + pEmp.getEmployeeInfo());
    }
}
```



2.2.1. Pertanyaan

1. Perhatikan class `Tester2` di atas, mengapa pemanggilan `e.getEmployeeInfo()` pada baris 8 dan `pEmp.getEmployeeInfo()` pada baris 10 menghasilkan hasil yang sama?

    - Variabel referensi `e` dan `pEmp` menyimpan **reference value** (alamat pointer memori) yang menunjuk ke instance objek yang persis sama di Heap (`PermanentEmployee`). Karena metode yang dipanggil adalah metode **overriding**, JVM melalui **dynamic method dispatch** mengeksekusi instruksi bytecode `PermanentEmployee.getEmployeeInfo()` yang terdaftar pada tabel metode virtual objek tersebut, sehingga menghasilkan luaran teks yang identik.

2. Mengapa pemanggilan method `e.getEmployeeInfo()` disebut sebagai virtual method invocation, sedangkan `pEmp.getEmployeeInfo()` tidak?

    - Pemanggilan `e.getEmployeeInfo()` disebut sebagai **virtual method invocation** karena terjadi divergensi antara tipe statis variabel saat kompilasi dengan tipe dinamis objek saat runtime: variabel `e` bertipe superclass `Employee`, namun objek aktual di Heap bertipe subclass `PermanentEmployee`. Kompiler mencatat **symbolic reference** milik kelas `Employee`, tetapi eksekusi metode aktualnya diputuskan secara dinamis (**late binding**) oleh JVM berdasarkan objek konkret `PermanentEmployee`. Sebaliknya, pada `pEmp.getEmployeeInfo()`, tipe deklarasi variabel dan tipe objek konkret di Heap adalah identik (`PermanentEmployee`), sehingga tidak melibatkan resolusi polimorfik antar-tingkat hierarki kelas.

3. Jadi apakah yang dimaksud dengan virtual method invocation? Mengapa disebut virtual?

    - **Virtual Method Invocation** (VMI) adalah mekanisme eksekusi metode **overriding** (instruksi bytecode `invokevirtual`) di mana penentuan metode aktual yang dijalankan didasarkan pada tipe kelas objek konkret penerima (**receiver object**) di Heap saat **runtime**, bukan tipe statis variabel deklarasinya saat **compile-time**. Disebut **virtual** karena pemetaan alamat instruksi metode tidak terikat secara absolut (**unbound / indirect**) saat kode dikompilasi, melainkan dihubungkan secara dinamis pada saat eksekusi

dengan membaca **virtual method table** (`vtable`) pada metadata kelas objek di Metaspace.

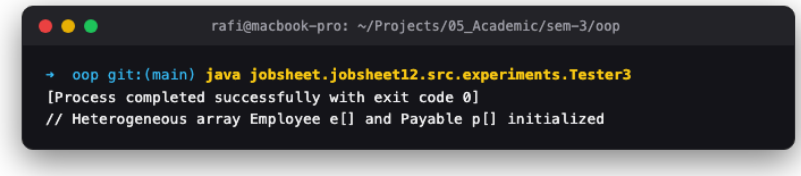
2.3. Percobaan 3: Heterogeneous Collection

Percobaan ketiga mengimplementasikan **Heterogeneous Collection**, yaitu pemanfaatan polimorfisme untuk menampung objek-objek dari kelas yang berbeda ke dalam satu struktur data array yang homogen di level tipe deklarasi. Dibuat array `Employee e[]` yang menampung objek `PermanentEmployee` dan `InternshipEmployee`, serta array `Payable p[]` yang menampung objek `PermanentEmployee` dan `ElectricityBill`.

```java
// Tester3.java
package jobsheet.jobsheet12.src.experiments;

public class Tester3 {
    public static void main(String[] args) {
        PermanentEmployee pEmp = new PermanentEmployee("Dedik", 500);
        InternshipEmployee iEmp = new InternshipEmployee("Sunarto", 5);
        ElectricityBill eBill = new ElectricityBill(5, "A-1");

        Employee e[] = {pEmp, iEmp};
        Payable p[] = {pEmp, eBill};
        // Employee e2[] = {pEmp, iEmp, eBill};
    }
}
```



2.3.1. Pertanyaan

1. Perhatikan array `e` pada baris ke-8, mengapa bisa diisi dengan objek-objek dengan tipe yang berbeda, yaitu objek `pEmp` (objek dari `PermanentEmployee`) dan objek `iEmp` (objek dari `InternshipEmployee`)?

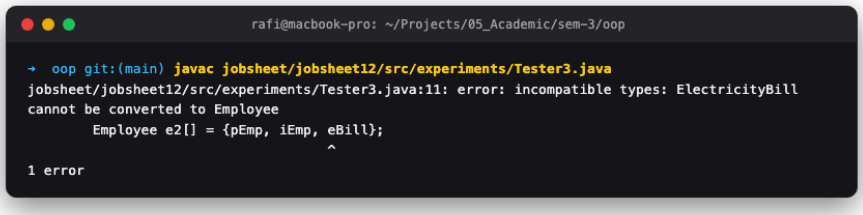
    - Array `e` dideklarasikan dengan tipe elemen referensi (**component type**) berupa superclass `Employee`. Di JVM, slot array objek di Heap dapat menyimpan **reference value** (pointer memori) dari instance kelas apa pun yang memiliki relasi turunan (**IS-A**) dengan `Employee`. Karena `PermanentEmployee` dan `InternshipEmployee` keduanya merupakan **subclass** langsung dari `Employee`, kompiler mengizinkan penyimpanan kedua pointer objek tersebut ke dalam array melalui mekanisme **implicit upcasting**. Struktur ini merepresentasikan konsep **Heterogeneous Collection**.

2. Perhatikan juga baris ke-9, mengapa array `p` juga bisa diisi dengan objek-objek dengan tipe yang berbeda, yaitu objek `pEmp` (objek dari `PermanentEmployee`) dan objek `eBill` (objek dari `ElectricityBill`)?

    - Array `p` dideklarasikan dengan tipe elemen antarmuka (**interface component type**) `Payable`. Di level sistem tipe Java, variabel atau slot array bertipe antarmuka dapat menampung **reference value** dari objek konkret apa pun yang mengimplementasikan (`implements`) kontrak antarmuka tersebut. Meskipun `PermanentEmployee` dan `ElectricityBill` berada pada hierarki kelas turunan yang terpisah, keduanya sama-sama mengimplementasikan interface `Payable`, sehingga sah disimpan bersama di dalam array `p`.

3. Perhatikan baris ke-10, mengapa terjadi error?

    - Kompilasi gagal dengan pesan error **type mismatch**: `error: incompatible types: ElectricityBill cannot be converted to Employee`



Penyebabnya adalah kegagalan **static type checking** oleh kompiler Java. Kompiler memverifikasi setiap elemen inisialisasi terhadap tipe dasar array (`Employee`). Objek `eBill` bertipe `ElectricityBill`, di mana kelas tersebut tidak mewarisi (**extends**) `Employee` pada graf hierarki pewarisan. Tanpa adanya relasi **IS-A**, konversi tipe referensi implisit (**upcasting**) tidak valid, sehingga baris kode tersebut ditolak saat kompilasi.

2.4. Percobaan 4: Polymorphic Arguments, instanceof, dan Object Casting

Percobaan keempat menerapkan **Polymorphic Arguments**, pemeriksaan tipe runtime melalui operator `instanceof`, dan **explicit object downcasting**. Dibuat class `Owner` yang memiliki method `pay(Payable p)` dan `showMyEmployee(Employee e)`.

Di dalam method `pay()`, dilakukan inspeksi RTTI via `instanceof` untuk mendeteksi apakah objek `p` merupakan instance dari `ElectricityBill` atau `PermanentEmployee`, kemudian di-**downcast** secara eksplisit untuk mengakses method khusus kelas konkret (`getBillInfo()`). Di method `showMyEmployee()`, dilakukan pengecekan serupa untuk menentukan kewajiban pembayaran gaji bulanan.

```
// Owner.java
package jobsheet.jobsheet12.src.experiments;

public class Owner {
    public void pay(Payable p) {
        System.out.println("Total payment = " + p.getPaymentAmount());
        if (p instanceof ElectricityBill) {
            ElectricityBill eb = (ElectricityBill) p;
            System.out.println("" + eb.getBillInfo());
        } else if (p instanceof PermanentEmployee) {
            PermanentEmployee pe = (PermanentEmployee) p;
            pe.getEmployeeInfo();
            System.out.println("" + pe.getEmployeeInfo());
        }
    }

    public void showMyEmployee(Employee e) {
        System.out.println("" + e.getEmployeeInfo());
        if (e instanceof PermanentEmployee) {
            System.out.println("You have to pay her/him monthly!!!");
        } else {
            System.out.println("No need to pay him/her :)");
        }
    }
}
```

```
// Tester4.java
package jobsheet.jobsheet12.src.experiments;

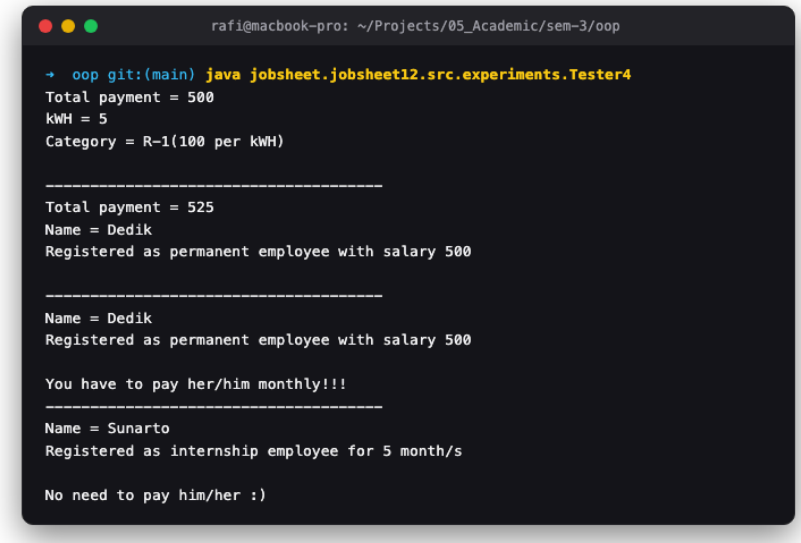
public class Tester4 {
    public static void main(String[] args) {
        Owner ow = new Owner();
```

```
        ElectricityBill eBill = new ElectricityBill(5, "R-1");
        ow.pay(eBill);
        System.out.println("---------------------------------------");

        PermanentEmployee pEmp = new PermanentEmployee("Dedik", 500);
        ow.pay(pEmp);
        System.out.println("---------------------------------------");

        InternshipEmployee iEmp = new InternshipEmployee("Sunarto", 5);

        ow.showMyEmployee(pEmp);
        System.out.println("---------------------------------------");
        ow.showMyEmployee(iEmp);
    }
}
```



2.4.1. Pertanyaan

1. Perhatikan class `Tester4` baris 7 dan baris 11, mengapa pemanggilan `ow.pay(eBill)` dan `ow.pay(pEmp)` bisa dilakukan, padahal method `pay()` memiliki argumen bertipe `Payable`?

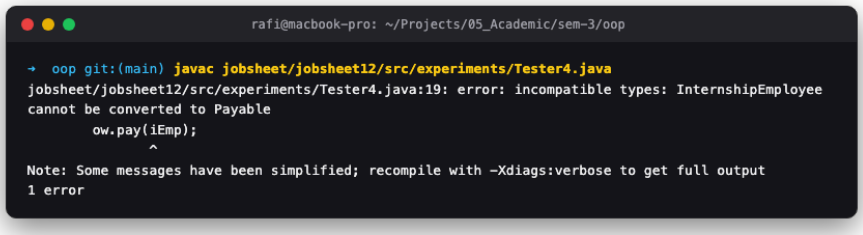
    - Ini merupakan penerapan **Polymorphic Argument**. Metode `pay(Payable p)` menerima parameter bertipe interface `Payable`. Karena `ElectricityBill` dan `PermanentEmployee` memenuhi kontrak antarmuka tersebut (`implements Payable`), kompiler Java secara otomatis melakukan **widening reference conversion** (**implicit upcasting**) pada saat **passing parameter**, sehingga alamat objek konkret di Heap dapat disalin secara aman ke dalam **local variable slot** milik frame metode `pay()`.

2. Jadi apakah tujuan membuat argumen bertipe `Payable` pada method `pay()` yang ada di dalam class `Owner`?

    - Tujuannya adalah menerapkan prinsip **abstraksi dan loose coupling** (**Open/Closed Principle**). Dengan menggunakan tipe antarmuka `Payable` sebagai tipe parameter formal, metode `pay()` dapat memproses berbagai macam objek transaksi yang dapat dibayar tanpa harus mengetahui implementasi detail kelas konkretnya. Hal ini mencegah terjadinya duplikasi kode atau penulisan metode **overloading** baru setiap kali ada penambahan jenis tagihan baru di masa depan.

3. Coba pada baris terakhir dari method `main()` yang ada di dalam class `Tester4` ditambahkan perintah `ow.pay(iEmp);`. Mengapa terjadi error?

    - Kompilasi gagal dengan pesan error **incompatible types**: `error: incompatible types: InternshipEmployee cannot be converted to Payable`



Penyebabnya adalah kelas `InternshipEmployee` hanya mewarisi superclass `Employee` dan **tidak mengimplementasikan interface** `Payable`. Karena objek `iEmp` tidak memenuhi kontrak tipe interface yang diminta oleh parameter formal `pay(Payable p)`, kompiler Java menolak pemanggilan tersebut saat analisis tipe statis (**static type checking**).

4. Perhatikan class `Owner`, diperlukan untuk apakah sintaks `p instanceof ElectricityBill` pada baris ke-6?

   - Sintaks `p instanceof ElectricityBill` mengeksekusi instruksi bytecode `instanceof` untuk melakukan **Runtime Type Identification** (RTTI). Pemeriksaan ini memvalidasi apakah instance objek di Heap yang dirujuk oleh pointer `p` benar-benar merupakan instansiasi dari kelas `ElectricityBill` (atau turunannya) sebelum dilakukan operasi **downcasting**. Pengecekan ini mutlak diperlukan untuk mencegah timbulnya **runtime exception** fatal berupa `ClassCastException`.

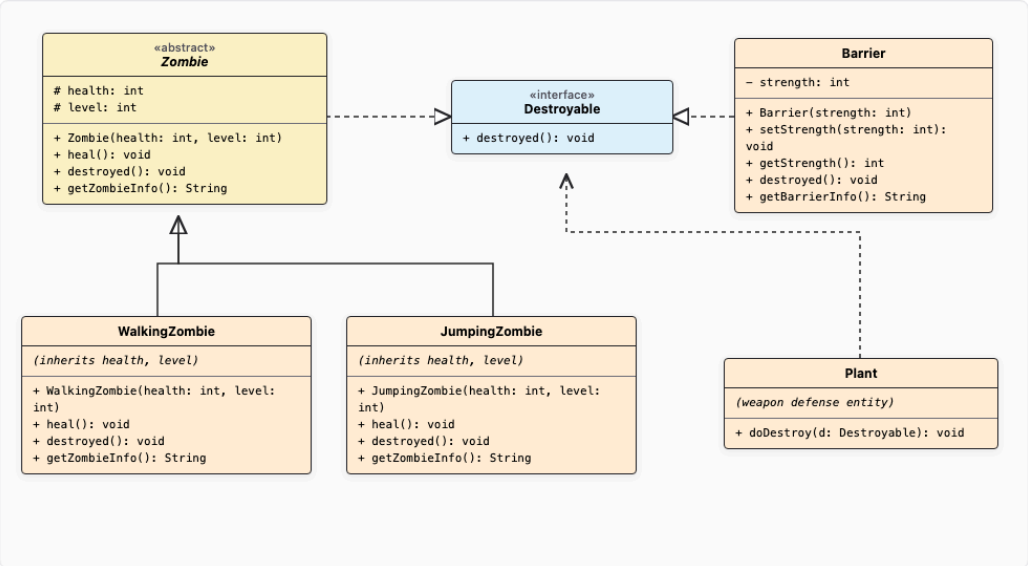
5. Perhatikan kembali class `Owner` pada baris ke-7, untuk apakah casting objek `ElectricityBill eb = (ElectricityBill) p` diperlukan? Mengapa objek `p` yang bertipe `Payable` harus di-cast ke dalam objek `eb` yang bertipe `ElectricityBill`?

   - Operasi tersebut merupakan **Explicit Downcasting** (instruksi bytecode `checkcast`). Variabel referensi `p` dideklarasikan dengan tipe antarmuka `Payable`, sehingga saat kompilasi, kompiler hanya mengizinkan pemanggilan metode yang terdaftar di dalam antarmuka `Payable` (yaitu `getPaymentAmount()`). Metode `getBillInfo()` merupakan metode spesifik milik kelas konkret `ElectricityBill`. Agar kompiler mengizinkan pemanggilan metode `getBillInfo()`, referensi polimorfik `p` harus diubah tipenya secara eksplisit (**downcast**) ke tipe kelas konkret `ElectricityBill`.

3. Tugas

3.1. Tugas Mandiri: Sistem Pertahanan Plants vs Zombies

Pada studi kasus tugas mandiri, dibangun simulasi sistem pertahanan game **Plants vs Zombies** dengan menerapkan arsitektur **interface-based polymorphism** dan pewarisan bertingkat. Sistem memodelkan entitas pertahanan tanaman (`Plant`) yang dapat menghancurkan berbagai macam target objek perusak (`WalkingZombie`, `JumpingZombie`, dan rintangan `Barrier`) melalui interface kontrak `Destroyable`.



Setiap target memiliki mekanisme penanganan kehancuran (**destruction logic**) dan penyembuhan (**healing logic**) yang spesifik:

1. **Interface `Destroyable`:** Mendefinisikan kontrak tunggal metode `public abstract void destroyed();`.
2. **Abstract Class `Zombie`:** Mengimplementasikan `Destroyable`, memiliki field **protected** `health` dan `level`, konstruktor `Zombie(health, level)`, metode abstrak `heal()`, `destroyed()`, serta `getZombieInfo()`.
3. **Class `WalkingZombie`:** Mewarisi `Zombie`. Pada saat dihancurkan (`destroyed()`), kesehatannya berkurang sebesar 20% secara pemotongan bilangan bulat (`health -= (int) (health * 0.2)`).
4. **Class `JumpingZombie`:** Mewarisi `Zombie`. Pada saat dihancurkan, kesehatannya berkurang sebesar 10% secara pemotongan bilangan bulat (`health -= (int) (health * 0.1)`).

5. **Class `Barrier`:** Mengimplementasikan `Destroyable` secara langsung. Memiliki field **private** `strength`. Setiap kali diserang (`destroyed()`), kekuatan ketahanannya berkurang tetap sebesar 9 poin (`strength -= 9`).
6. **Class `Plant`:** Memiliki metode destruksi polimorfik `public void doDestroy(Destroyable d) { d.destroyed(); }`. Melalui metode tunggal ini, tanaman dapat menghancurkan objek apa pun yang memenuhi kontrak `Destroyable` tanpa memerlukan **overloading**.

```
// Destroyable.java & Zombie.java
package jobsheet.jobsheet12.src.assignments;

public interface Destroyable {
    abstract void destroyed();
}

public abstract class Zombie implements Destroyable {
    protected int health;
    protected int level;

    public Zombie(int health, int level) {
        this.health = health;
        this.level = level;
    }
    public abstract void heal();
    public abstract void destroyed();
    public abstract String getZombieInfo();
}
```

```
// WalkingZombie.java & JumpingZombie.java
package jobsheet.jobsheet12.src.assignments;

public class WalkingZombie extends Zombie {
    public WalkingZombie(int health, int level) {
        super(health, level);
    }

    @Override
    public void heal() {
        if (health >= 100) return;
        if (this.level == 1) this.health += (int) (this.health * 0.1);
        else if (this.level == 2) this.health += (int) (this.health * 0.3);
        else if (this.level == 3) this.health += (int) (this.health * 0.4);
    }

    @Override
    public void destroyed() {
        if (this.health <= 0) return;
```

```
        this.health -= (int) (this.health * 0.2);
    }

    @Override
    public String getZombieInfo() {
        return "Walking Zombie Data = \nHealth = " + this.health + "\nLevel = " +
this.level + "\n";
    }
}

public class JumpingZombie extends Zombie {
    public JumpingZombie(int health, int level) {
        super(health, level);
    }

    @Override
    public void heal() {
        if (this.health >= 100) return;
        if (this.level == 1) this.health += (int) (this.health * 0.3);
        else if (this.level == 2) this.health += (int) (this.health * 0.4);
        else if (this.level == 3) this.health += (int) (this.health * 0.5);
    }

    @Override
    public void destroyed() {
        if (this.health <= 0) return;
        this.health -= (int) (this.health * 0.1);
    }

    @Override
    public String getZombieInfo() {
        return "Jumping Zombie Data = \nHealth = " + this.health + "\nLevel = " +
this.level + "\n";
    }
}
```

```
// Barrier.java, Plant.java, & Tester.java
package jobsheet.jobsheet12.src.assignments;

public class Barrier implements Destroyable {
    private int strength;

    public Barrier(int strength) { this.strength = strength; }
    public int getStrength() { return strength; }
    public void setStrength(int strength) { this.strength = strength; }

    @Override
    public void destroyed() { this.strength -= 9; }

    public String getBarrierInfo() {
```

```
        return "Barrier Strength = " + this.strength;
    }
}

public class Plant {
    public void doDestroy(Destroyable d) {
        d.destroyed();
    }
}

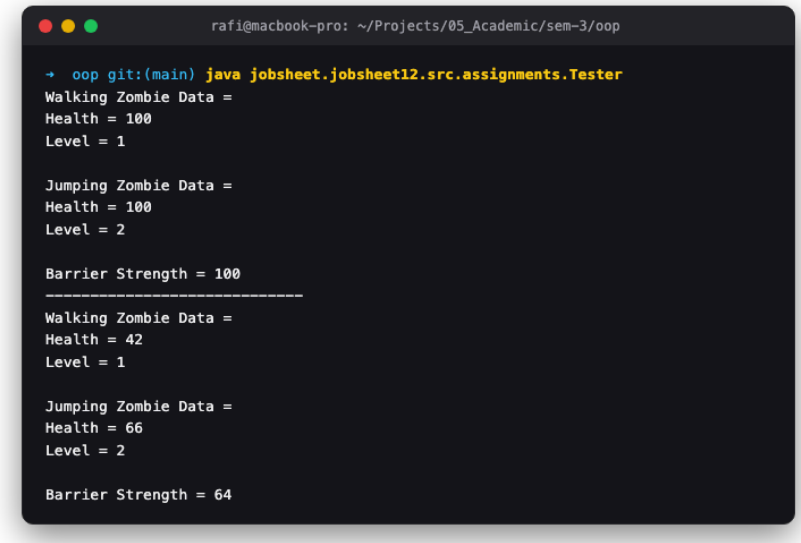
public class Tester {
    public static void main(String[] args) {
        WalkingZombie wz = new WalkingZombie(100, 1);
        JumpingZombie jz = new JumpingZombie(100, 2);
        Barrier b = new Barrier(100);
        Plant p = new Plant();

        System.out.println("" + wz.getZombieInfo());
        System.out.println("" + jz.getZombieInfo());
        System.out.println("" + b.getBarrierInfo());

        System.out.println("-----------------------------");

        for (int i = 0; i < 4; i++) {
            p.doDestroy(wz);
            p.doDestroy(jz);
            p.doDestroy(b);
        }

        System.out.println("" + wz.getZombieInfo());
        System.out.println("" + jz.getZombieInfo());
        System.out.println("" + b.getBarrierInfo());
    }
}
```



Berdasarkan hasil eksekusi program di atas, simulasi penyerangan 4 siklus berturut-turut menghasilkan reduksi nilai yang akurat sesuai target output modul praktikum:

- **Walking Zombie (awal: 100):** Mengalami reduksi 20% berturut-turut: 100 $\rightarrow$ 80 $\rightarrow$ 64 $\rightarrow$ 52 $\rightarrow$ **42**.
- **Jumping Zombie (awal: 100):** Mengalami reduksi 10% berturut-turut: 100 $\rightarrow$ 90 $\rightarrow$ 81 $\rightarrow$ 73 $\rightarrow$ **66**.
- **Barrier (awal: 100):** Mengalami pengurangan 9 poin per siklus: $100 - (9 \times 4) =$ **64**.